

Time Series Forecasting: An Exploratory Study Using S&P 500 ETF Data

Gina Occhipinti

Table of contents

1	Abstract	2
2	Introduction	2
3	Literature Review	3
4	Methodology	5
4.1	Data Preparation	5
4.1.1	Load Data	6
4.1.2	Transform Data and Check Missing Values	6
4.1.3	Basic Plot	7
4.1.4	Point Plot	7
4.1.5	Seasonality Plot	8
4.1.6	Autocorrelation Plot	9
4.1.7	ACF First Differencing	10
4.1.8	Inflation-Adjusted Data	11
4.1.9	Inflation-Adjusted Plot	12
4.1.10	Log and Cube Root Transformations	13
4.1.11	STL Decomposition	15
4.1.12	Plot Trend Line to Data	16
4.1.13	STL Decomposition Features	17
4.2	Model Development	18
4.2.1	Train-Test Split	18
4.2.2	Create Models	19
4.2.3	Create Forecasts	20
4.3	Model Evaluation	20
4.3.1	Forecast Plot	20
4.3.2	Performance Metrics	21

4.3.3	Residual Diagnostics	22
4.3.4	Innovation Residuals	23
4.3.5	Residual Histogram	25
5	Results and Analysis	26
6	Discussion	27
7	Conclusion	28
8	References	28

1 Abstract

This project is an exercise studying the applications of time series forecasting techniques to predict financial stock prices. Investors looking to maximize profit from financial investing can benefit from data-driven approaches to estimating future stock prices and forecasting techniques can be a part of such approaches. This project particularly uses 10-year monthly price data to forecast the SPY-USA ETF price using the following time series forecasting models: exponential smoothing (ETS), drift (random walk), naïve, seasonal naïve, and an ensemble method.

2 Introduction

Forecasting is a valuable tool in producing intelligent ideas about future occurrences. This is especially pertinent in the finance industry, a key industry of the United States economy. Understanding the movement and being able to predict the future values of financial instruments like stocks, bonds, and exchange traded funds (ETFs) is extremely valuable for those looking to profit off these instruments. Yet this task is challenging because their movements can exhibit volatility. The efficient market hypothesis states that markets have perfect or near perfect knowledge and can adjust to create positions (buying and selling of securities) that end up neutralizing any financial benefit. However, stock mispricings still occur. Predicting individual stocks is one of the most difficult endeavors in modern financial because of sensitivity of stocks to current events. ETFs are advantageous because they diversify investments within respective asset classes. This is good during economic uncertainty, where investors seek a dependable income and hedge against volatility (Ngwaba, 2025). In recent years, there has been a shift towards buying ETFs due to their benefits, with market caps surpassing USD 10 trillion in 2023. ETF benefits include low management fees, high liquidity, and broad market exposure (Shih et al., 2024).

It can be advantageous to take certain positions based on suspected market movements (i.e., buy low and sell high to maximize profit). Knowing *when* to take a certain position can be exceedingly valuable and financial forecasting can be an interesting tool for this task. Still academic studies show mixed results on whether past trends predict future returns (Yates, 2024).

A common ETF Americans invest in is the SPY (under the ticker SPY-USA). The SPY tracks a market cap-weighted index of US large- and mid-cap stocks selected by the S&P Committee. This ETF is suitable to invest in long-term with a passive investing approach, because it tracks the popular S&P 500 US index. The S&P 500 US index represents the US market and diversifies an investment across many companies, reducing risk exposure. Being able to predict key aspects of the SPY-USA, such as the daily closing price, is valuable for investors looking to better time their positions.

Several different forecasting strategies can be useful for financial data. These are typically non-seasonal methods, as stock price data does not follow a seasonal pattern and is more erratic. Hyndman and Athanasopoulos (2021) outline the following models as examples of simple but effective techniques in forecasting financial stock price data:

- Mean model: The forecasts of all future values are equal to the average (or “mean”) of the historical data
- Naïve model: All forecasts are the value of the last observation
- Drift model: A variation on the naïve method that allows the forecasts to increase or decrease over time, where the amount of change (called the drift) is the average change seen in the historical data

These simple models are often the benchmark in evaluating other time series modeling approaches.

This analysis will explore some of these methods for forecasting SPY price data, along with additional methods, such as an exponential smoothing (ETS), seasonal naïve (snaïve), and an ensemble model that combines the forecasts of these models using averaging. This analysis will also consider external research applying other time series and machine- and deep learning-based forecasting methods and weigh the implications of these methods for stock price applications.

3 Literature Review

Literature on forecasting methods applied to financial data is vast. There are basic theories on how to predict market performance. Yates (2024) discusses one theory related to momentum. Rooted in behavioral finance, the assumption is that market movements will continue in the same direction, but only in the short-term. Over longer periods, this effects reverses. Another theory is that market prices are martingales, or a mathematical series where the best prediction

for the next number is the current number. This is essentially the idea behind naïve forecasting models, where the forecast takes on the previous observation’s value.

Academic studies have investigated ways to predict the market using both traditional time series techniques, but also machine and deep learning models, with mixed results. Pande and Kumar (2025) did a study “Forecasting Stock Indices: Stochastic and Artificial Neural Network Models” acknowledging traditional methods to forecast stock prices and indices such as stochastic time series models, autoregressive integrated moving average (ARIMA), and jump diffusion models. They also looked at modern machine learning methods such as feed forward networks (FFN) and long short-term memory (LSTM) models. This study creates models used to forecast the prices of major stock indices in both developed and emerging markets 10, 20, and 30 days ahead. The study used adjusted closing price as the main variable to predict. The metrics used to evaluate performance included root mean squared error (RMSE), mean absolute error (MAE), and mean absolute percentage error (MAPE), metrics this forecasting exercise paper will also use to assess model performance and fit of techniques to the data. The Pande and Kumar study found that LSTM models performed best for this forecasting application over almost all indices over all time horizons *except* for the S&P 500 and Nikkei 225 10 days ahead. LSTM solves the problem of long-term dependence arising in temporal data and has feedback connections which allow the networks to decipher hidden patterns in the temporal data more effectively (Pande and Kumar, 2025). For these indices specifically, the autoregressive fractionally integrated moving average (ARFIMA) time series model performed best. ARFIMA can make up for shortcomings of long-range dependent data, though the model has an almost linear fit and cannot model inevitable shocks in price.

The authors question whether prediction accuracy alone can imply economic gains. Historical price data is not always a perfect indicator of future price movements as there are other factors (i.e., macroeconomic conditions, geopolitical events) that can affect their movement. Even with effective and accurate LSTM models, high-frequency trading requires high computational power, which is difficult for most average investors to obtain. Another limitation of this study is that it doesn’t take into account cross-correlation between two different stock indices which affects stock price movement as well (Pande and Kumar, 2025).

Shih et al. (2024) wrote a journal article “Forecasting ETF Performance: A Comparative Study of Deep Learning Models and the Fama-French Three-Factor Model” comparing deep learning models to more traditional models, such as the Fama-French 3-factor models. The Fama-French models are expanded versions of the capital asset pricing model (CAPM) that tries to explain the risk and return relationship. Factors like market risk premium, book-to-market ratio, and size factor can affect the accuracy of predictions. This study seeks to enhance the investment strategy toolkit by comparing the performance of these traditional models against modern deep learning methods (Shih et al., 2024).

Particularly, this study looks at the return prediction of Taiwanese ETFs, using MAE as a metric to compare the performance of artificial neural networks (ANNs) and linear models. Similar to the study above, it found that LSTM has superior performance, particularly in

combination with the Fama-French 3-factor model. This combination was effective at processing complex temporal patterns in daily ETF returns and integrating multifaceted variables. Study limitations include the risk of overfitting to the data with complex neural networks like LSTM and also focusing solely on MAE to assess performance, rather than other metrics like MSE or RMSE. Additionally, the focus on a Taiwanese dataset limits generalizing the findings to other markets.

Another study, “Forecasting Covered Call Exchange-Traded Funds (ETFs) Using Time Series, Machine Learning, and Deep Learning Models” by Ngwaba (2025) takes a unique approach, being one of the first to focus on forecasting applications specifically for covered call ETFs. Covered call ETFs (like QYLD, JEPI) use a strategy of covered call writing to generate income for investors. It entails selling call options on the underlying securities owned by the investor in exchange for premiums while still allowing for potential price appreciation up to the strike price of options (Ngwaba, 2025). It compares the ARIMA and heterogeneous autoregressive (HAR) models with advanced machine learning techniques like random forest, support vector regression, convolutional neural networks (CNNs), and recurring neural networks (RNNs).

The study found that deep learning models are effective at identifying nonlinear patterns and temporal dependencies in price movements of covered call ETFs, outperforming traditional time series and machine learning techniques. Specifically, RNNs consistently outperformed every other model in the study, having the lowest MAPE. A limitation with this study is that deep learning models require significant computational demands and extensive data needs. The author suggests future research to explore ensemble techniques, hybrid approaches, and alternative deep learning frameworks like transformers to boost forecasting precision and dependability. Also, it suggests integrating a range of indicators like technical metrics and macroeconomic factors (Ngwaba, 2025).

The analysis in this forecasting exercise paper focuses on simple models, addressing some of the gaps noted in existing literature around the impracticality of high-computational models, especially for the average everyday investor. Simpler models are more accessible, especially with the availability of historical ETF price data. The models in this study include a naïve forecasting method, which aligns with existing theories of financial data patterns (e.g., martingale theory). This study also explores an ensemble method, something noted as an area worthy of further exploration by Ngwaba. This study specifically looks at the SPY-USA, representing the S&P 500 and can generalize well to other markets as U.S. financial performance has significant global influence and attracts many investors.

4 Methodology

4.1 Data Preparation

For this investigation, historical monthly price values for the SPY-USA: SPDR S&P 500 ETF Trust were selected. This dataset was pulled from FactSet, a company that provides financial

data and analytics tools to investment professionals and universities. The dataset contains 10 years of historical data from 2016 to 2025. Variables in the original dataset included Open, Low, High and %Change. For simplicity, the variables of interest were reduced to the temporal variable (month and year) and the price, referring to the closing price or the final traded price when the market closed that day.

Preparing this dataset involved cleaning it for time series analysis, including mutating the temporal variable to be in monthly format, and identifying possible missing values. Preparation also involved visualizing the data through decomposition to detect the data's true patterns, like trends or seasonality. This exploration and preparation exercise provides a deeper understanding of the data to support model selection and interpret model performance.

4.1.1 Load Data

```
# import SPY-USA price history
data <- read.csv("PriceHistory.csv")
```

4.1.2 Transform Data and Check Missing Values

```
# change character date to date format
data$Date <- lubridate::mdy(data$Date)

# convert date to month and year format
data <- data |>
  mutate(Date = yearmonth(Date))

# turn data into a tsibble
data <- data |>
  as_tsibble(index = Date)

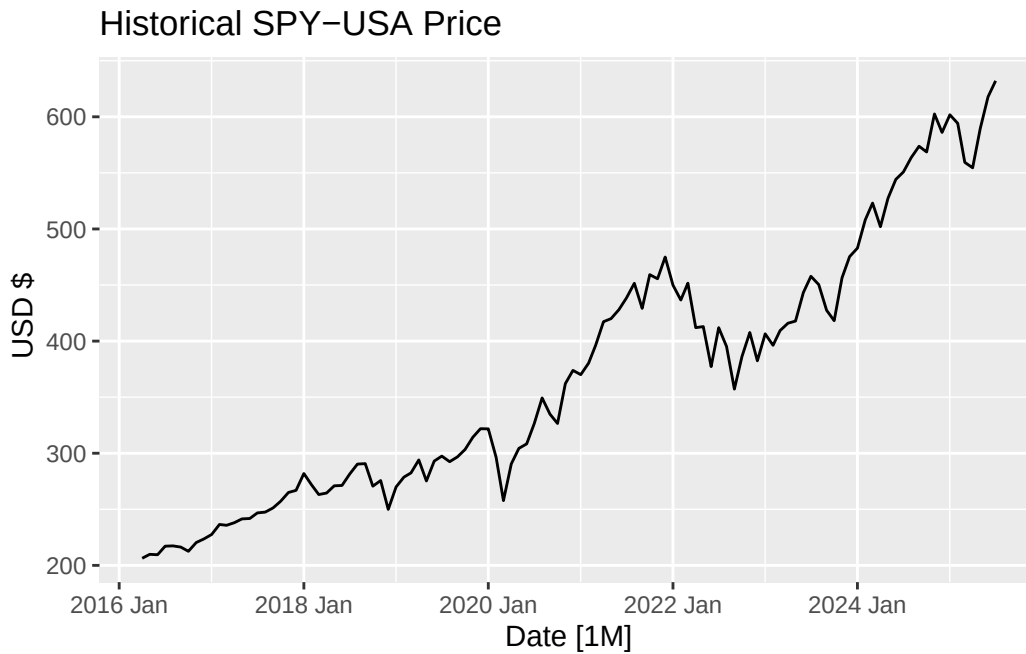
# select pertinent columns - just price and date
data <- data |>
  select(Date, Price)

# check for missing values
sum(is.na(data))
```

Below is a chart visualizing the last 10 years of SPY-USA closing price data.

4.1.3 Basic Plot

```
# create basic plot
data |>
  autoplot(Price) +
  labs(y = " USD $", title = "Historical SPY-USA Price")
```

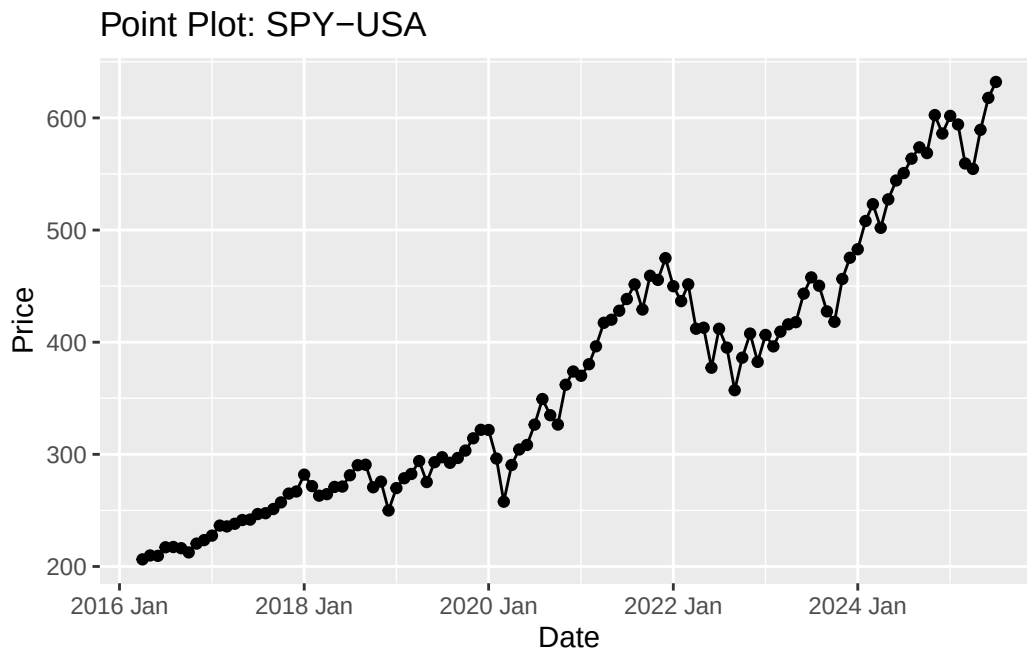


The visualization shows there is a clear upward trend, where the stock price data has been increasing over the past 10 years, particularly between 2020 and 2022 and again between 2022 and 2025. The visualization also indicates that the data is not seasonal or repeat at fixed intervals. This is expected behavior of stock price performance data. There are some possible cyclical patterns, where the chart shows some troughs (like the post-COVID-19 drop). These are likely related to the business cycle, and are expected as the S&P 500 tracks company performance reflective of market performance.

To better visualize some of the peaks and troughs in the data, a point plot was used to see where the exact changes in price occurred throughout the months.

4.1.4 Point Plot

```
# create point plot
data |>
  ggplot(aes(x=Date, y=Price)) +
  geom_line() +
  geom_point() +
  labs(title = "Point Plot: SPY-USA")
```



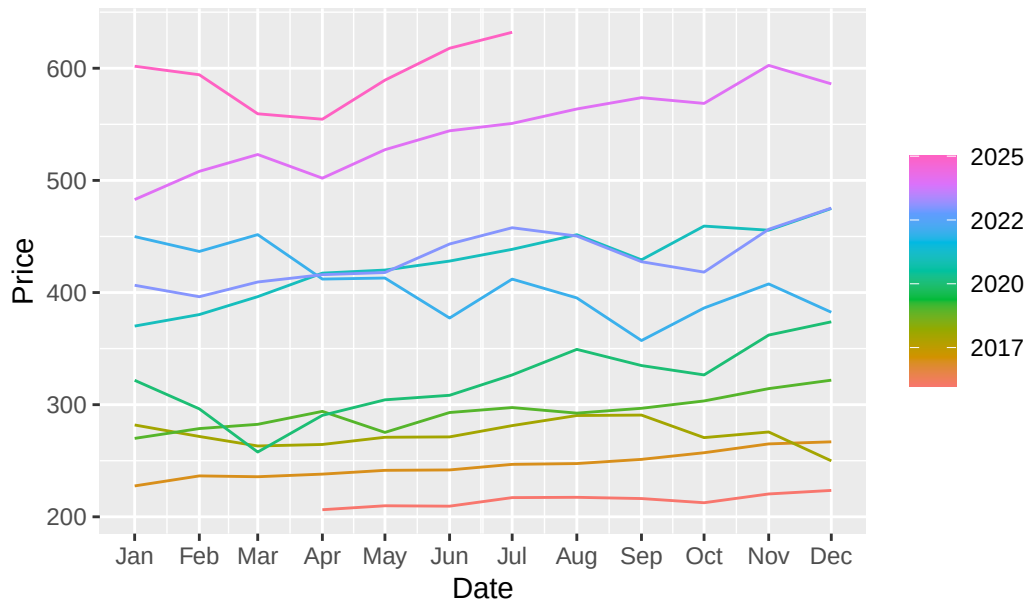
This visualization further confirms the major dips in price - just before 2019, just after 2020, and during 2022.

Next, a seasonality plot was created to further detect possible seasonal patterns in the data.

4.1.5 Seasonality Plot

```
# create seasonality plot
data |>
  gg_season(Price) +
  labs(title = "Seasonality Plot: SPY-USA")
```

Seasonality Plot: SPY-USA

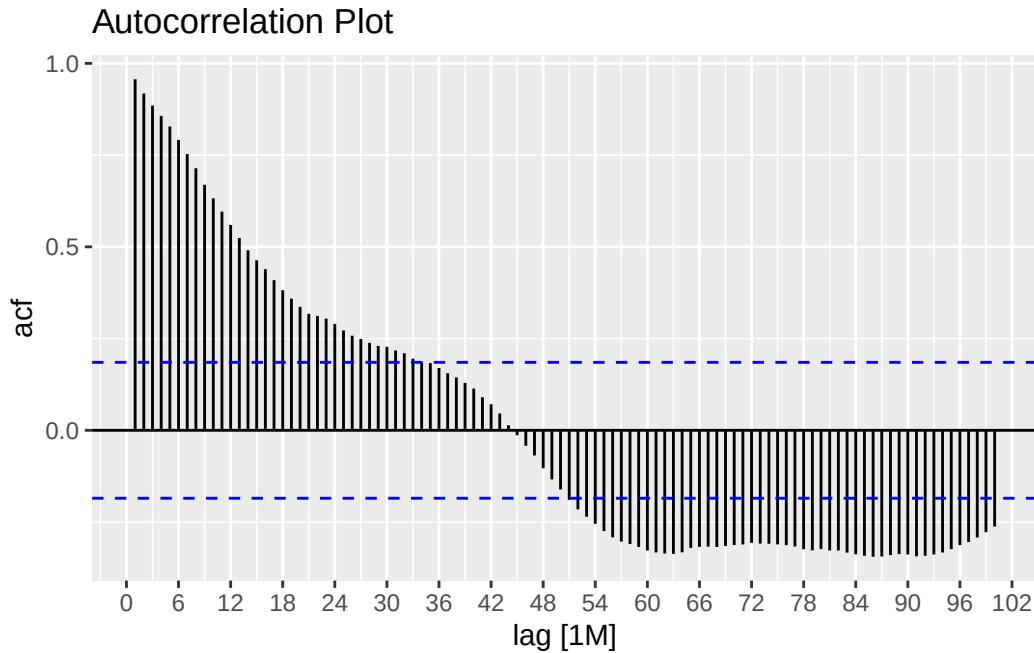


This plot doesn't reveal any major seasonal patterns. Rather, the data pattern has sporadic drops and raises through the months. This helps validate a lack of seasonality in the raw data.

4.1.6 Autocorrelation Plot

To understand if there are autocorrelations between observations, an autocorrelation (ACF) plot was used on the data, which shows correlations between any two observations in the time series.

```
# test autocorrelations
data |>
  ACF(Price, lag_max = 100) |>
  autoplot() +
  labs(title = "Autocorrelation Plot")
```



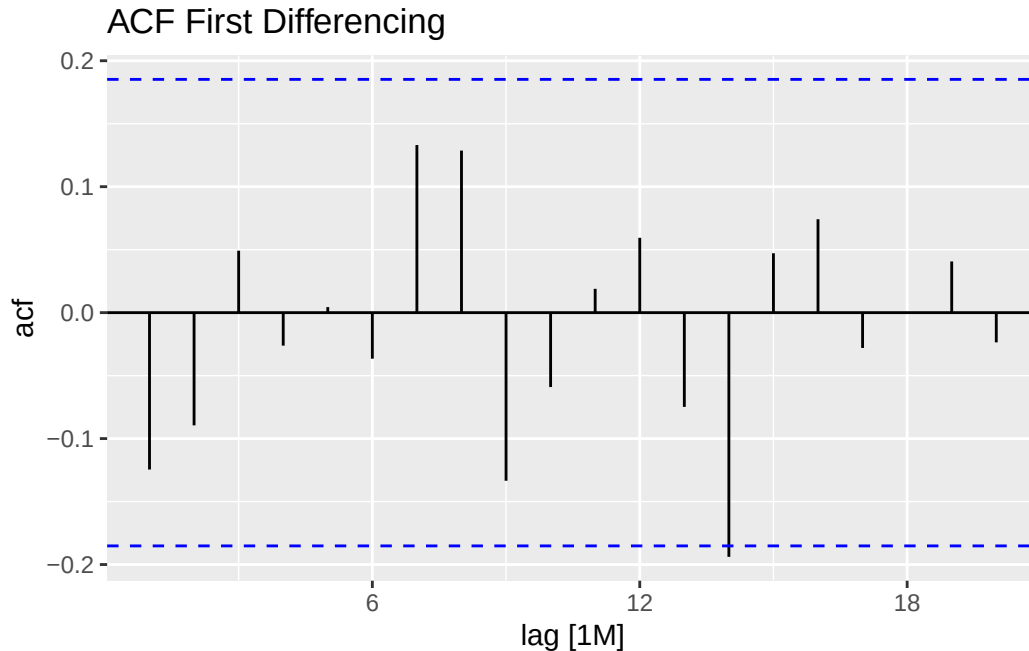
The gradual patterns visualized in this ACF plot is intuitive for ETF data - today's ETF closing price is similar to yesterday's closing price. The closing price 6 months out will be much more related to today's value than the closing price 36 months out. Barring any major shocks from current events, stock price data is typically consistent in the short-term while also exhibiting random noise. This consistency or autocorrelation decays over time with the lag, gradually approaching zero, until the pattern reverses to negative correlations. This is a classic signature of non-stationary data with a persistent trend, as smaller lags tend to be large and positive, indicating the data has a long-term "memory" of where the closing price has been and is drifting upwards over time. This aligns well with the momentum theory discussed in the literature by Yates (2024).

For modeling, autocorrelation patterns left unaddressed will produce correlated residuals, causing misleading, biased results. To address this, I can apply a first differencing approach to alter the variable being forecasted, or use models that can handle these autocorrelations innately.

4.1.7 ACF First Differencing

Time series data should be stationary or stable over time for modeling because it is predictable, and has mean, variance, and autocovariances that do not change over time. Stationarity helps account for the shocks exhibited in financial price data by making them consistent over time so models are better able to quantify the range of probable outcomes. To make the data stationary, a first differencing approach was applied, shifting the focus from the absolute price level to the inter-day price changes.

```
# test autocorrelations on differenced data
data |>
  mutate(diff_price = difference(Price)) |>
  ACF(diff_price) |>
  autoplot() +
  labs(title = "ACF First Differencing")
```



While the initial ACF plot showed autocorrelations, the second ACF plot of the first-differenced data, shows no significant autocorrelation and thus achieving stationarity. This confirms that the raw data did indeed follow a random walk pattern with drift captured as the trend - typical movement of financial stock data.

An ETS model with an additive trend component is well-suited for this data, as its state-space framework internally accounts for non-stationarity without requiring manual pre-processing of the series.

4.1.8 Inflation-Adjusted Data

Interestingly, some of the biggest increases in price align with increased inflation in the U.S. from 2022 to 2025. Inflation-adjusted ETF prices may present a more accurate picture of SPY-USA price movement and trends. Monthly data from FRED on Core CPI, which is the CPI for all urban consumers less food and energy due to price volatility, was used to adjust the data.

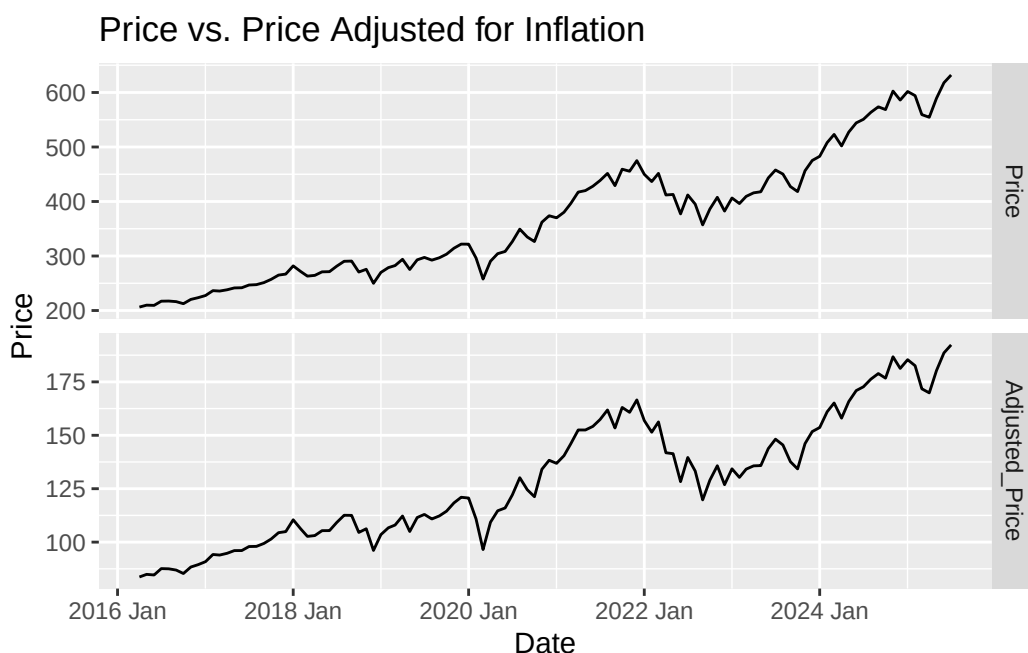
```
# adjust for inflation
# monthly Core CPI - CPI all urban consumers all items less food and energy

# load CPI dataset
cpi_data <- read_csv("CPILFESL.csv")
```

4.1.9 Inflation-Adjusted Plot

```
# prep to join with data
cpi_data <- cpi_data |>
  mutate(Date = yearmonth(observation_date)) |>
  select(-observation_date)

# join data, reshape, and visualize adjusted vs. non-adjusted
data |>
  left_join(cpi_data, by = "Date") |>
  mutate(Adjusted_Price = Price / CPILFESL * 100) |> # CPI has 100 index
  pivot_longer(c(Price, Adjusted_Price),
    values_to = "Price") |>
  mutate(name = factor(name,
    levels = c("Price", "Adjusted_Price"))) |>
  ggplot(aes(x = Date, y = Price)) +
  geom_line() +
  facet_grid(name ~ ., scales = "free_y") +
  labs(title = "Price vs. Price Adjusted for Inflation")
```

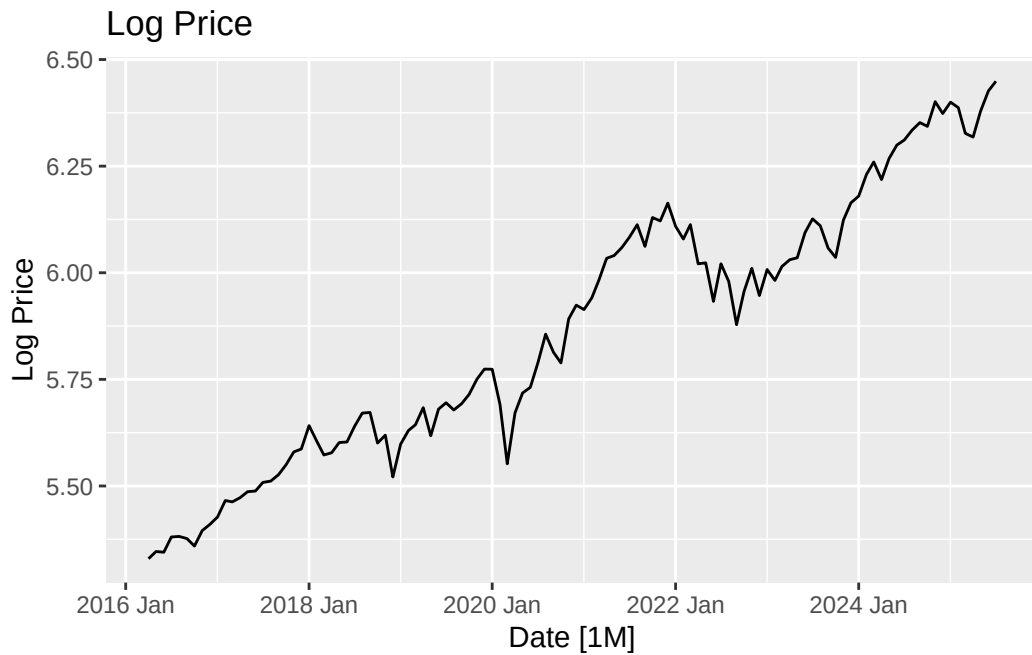


Overall, the adjusted ETF price data follows the same trend and cyclical patterns as the non-adjusted data. Of course, the adjusted price is lower than the inflated price, but its pattern is not significantly different. The adjusted dollars are for 1982 - 1984 when the core CPI was 100.

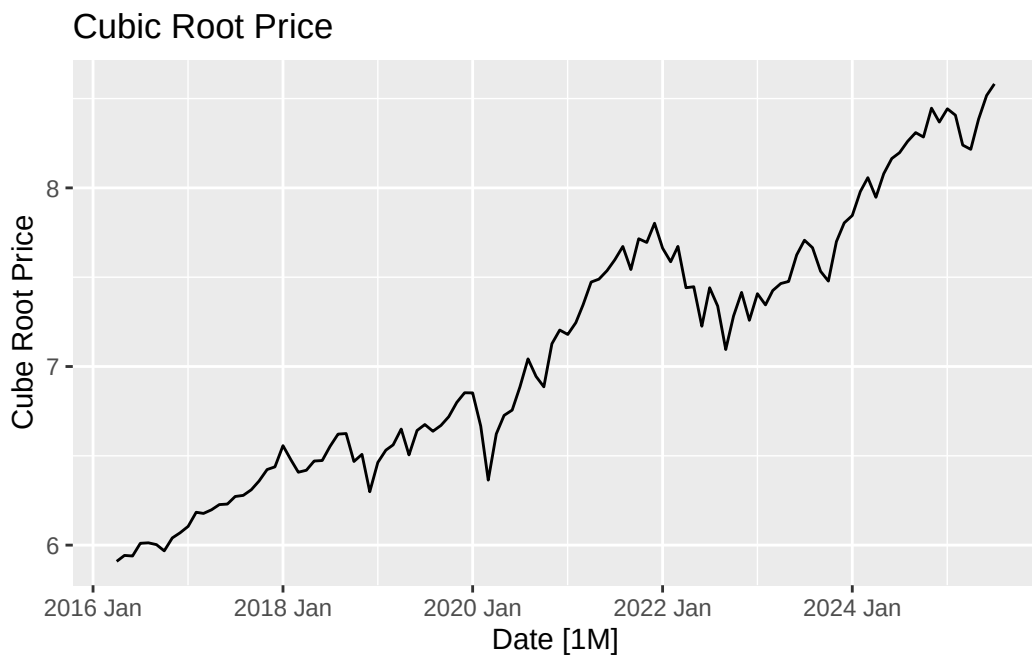
4.1.10 Log and Cube Root Transformations

To better decompose the time series data and visualize its data patterns, common math-based transformations can also be applied like log or cube root transformations. These help stabilize the variance or swings in the data.

```
# try log of price
data |>
  autoplot(log(Price)) +
  labs(title = "Log Price", y = "Log Price")
```



```
# try cube root of price  
data |>  
  autoplot(Price^(1 / 3)) +  
  labs(title = "Cubic Root Price", y = "Cube Root Price")
```



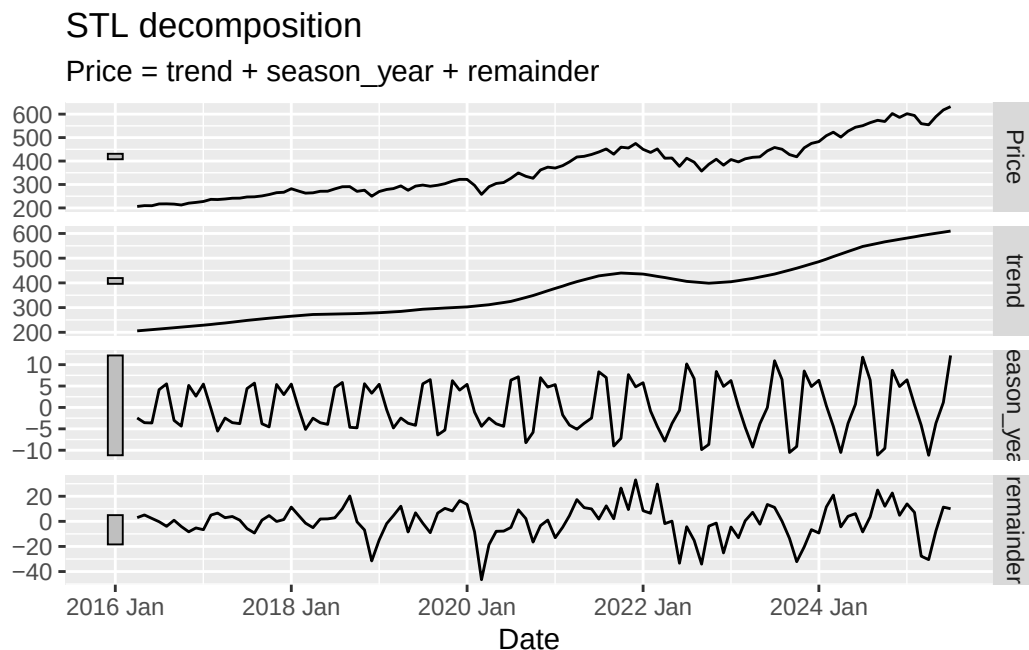
It appears that the log and cubic transformations don't cause significant change in the data patterns. Viewing the plots of the SPY-USA price history, the fluctuations do not appear to be growing exponentially and the data appears to have a constant variance (not swinging wildly), so this makes intuitive sense. A Box-Cox transformation was omitted as the series exhibits homoscedasticity (constant variance), meaning there is no systematic increase in volatility as the price level rises. Using transformations that aren't needed can also negatively impact forecast prediction intervals so they were not used further in modeling.

4.1.11 STL Decomposition

Next, the data was further decomposed with seasonal-trend decomposition using LOESS (STL) decomposition plots to break down the price data.

```
# create STL decomposition
dcmp <- data |>
  model(stl = STL(Price))

# get components
components(dcmp) |>
  autoplot()
```



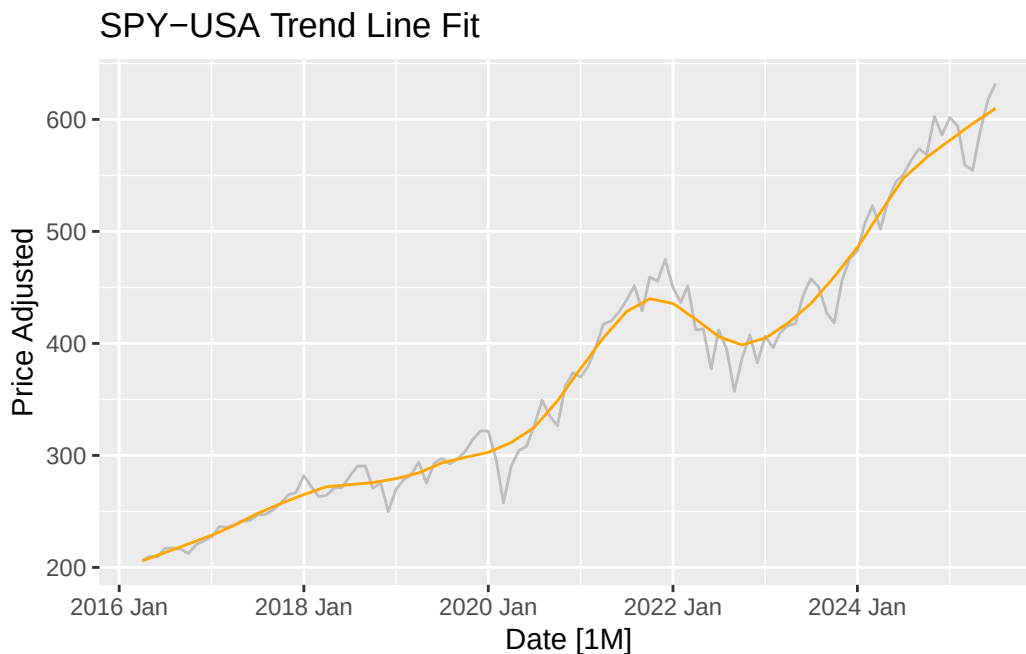
This STL decomposition is useful in stripping out each component of the data - trend, seasonality and the remainder. The plot shows a smooth upward trend. Surprisingly, the plot

appears to show seasonality in the year-to-year breakdown, with a multiplicative effect, where the magnitude of price fluctuations increases alongside the trend. Sometimes STL decomposition can force seasonality even if the raw data or data context would implicate there is no seasonality. The remainder has lots of white noise.

To see how well the trend captures the pattern in the raw data, the trend component was plotted against the actual data.

4.1.12 Plot Trend Line to Data

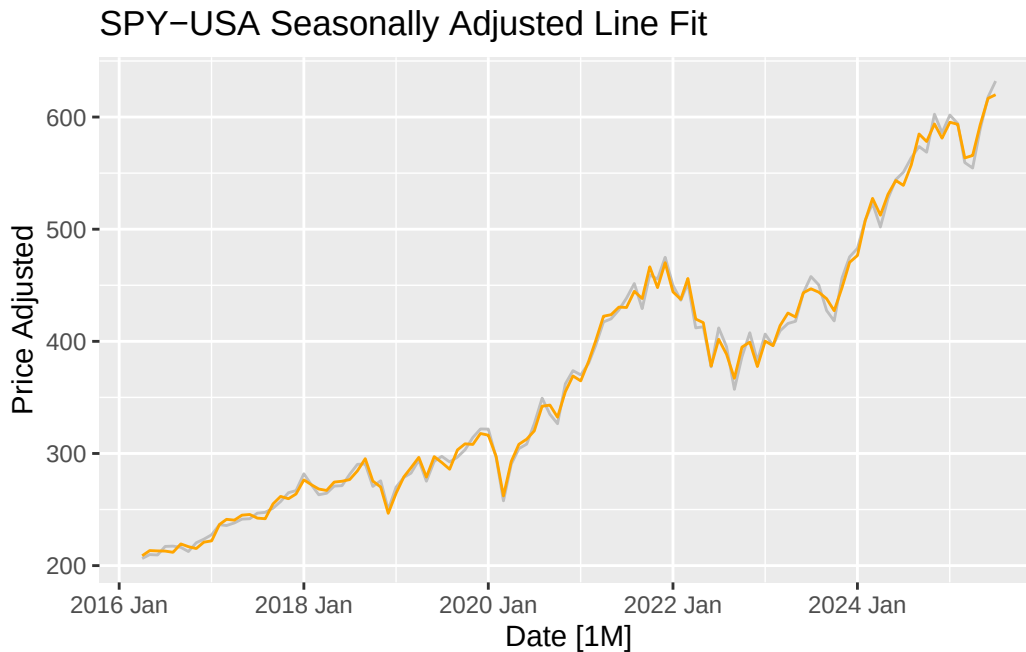
```
# compare decomposed trend to data
data |> autoplot(Price, color = "gray") +
  autolayer(components(dcmp), trend, color = "orange") +
  labs(y = "Price Adjusted", title = "SPY-USA Trend Line Fit")
```



When accounting for seasonality by removing it, the trend and remainder aligns more closely with the raw series, as it preserves the price's remainder while filtering out non-predictable seasonal fluctuations.

```
# plot decomposed seasonally adjusted data
data |>
  autoplot(Price, color = "gray") +
```

```
autolayer(components(dcmp), season_adjust, color = "orange") +
labs(y = "Price Adjusted", title = "SPY-USA Seasonally Adjusted Line Fit")
```



This helps confirm that though there is seasonality in the STL plot, it might not be a reliable signal, and could instead be an aspect of the STL decomposition functionality forcing seasonality.

4.1.13 STL Decomposition Features

To confirm the strong trend and low seasonality, STL decomposition features were analyzed.

```
# pull STL features
data |>
  features(Price, feat_stl) |>
  pivot_longer(cols = everything(),
               names_to = "Feature",
               values_to = "Value") |>
  kable(digits = 3,
        caption = "STL Decomposition Features",
        booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "hold_position"))
```

Table 1: STL Decomposition Features

Feature	Value
trend_strength	0.986
seasonal_strength_year	0.192
seasonal_peak_year	4.000
seasonal_trough_year	1.000
spikiness	8.999
linearity	1160.538
curvature	172.150
stl_e_acf1	0.436
stl_e_acf10	0.298

STL decomposition features can reveal insights about how strong the trend or the seasonality is or measure the prevalence of spikes in the remainder component. From the table output, the trend strength is close to 1.0, indicating the trend explains almost all of the movement. The seasonal strength year is low, indicating that the data doesn't follow a calendar-based of fixed pattern. This helps validate the plots above where the seasonally-adjusted line follows the raw price more closely because it retains the irregular volatility of the market.

While the STL visualization suggests a multiplicative seasonal relationship, given that the seasonal strength is negligible at 0.192, adding a multiplicative seasonal component might be “over-modeling” noise. However, for research purposes, a multiplicative Holt-Winters model was tested against an additive trend model to determine if accounting for multiplicative seasonality improved predictive accuracy.

4.2 Model Development

In order to develop and the test the various forecasting models, the data was split into 80% training and 20% testing data.

4.2.1 Train-Test Split

```
# split data - 20%
test_data <- data |>
  filter_index("2023 Jan" ~ .)

# split data - 80%
```

```
train_data <- data |>
  filter_index("2015 Jul" ~ "2022 Dec")
```

The following models were used in this analysis to forecast the SPY-USA closing price:

- Naïve Model: Forecast equals the most recent observation
- Seasonal Naïve Model: Forecast equals the observation value from the same season in the previous year
- Drift (Random Walk): Forecast is based on the last observation plus a constant average change or drift, adjusted for random error
- ETS Models (Auto-selected): Exponential Smoothing State Space Model with automated parameter selection.
- Ensemble Model: Combine forecasts of the models using averaging.

First, the `model` function was applied to the data and added to each corresponding model. This created a mable, or a model table, where each cell corresponds to a fitted model. The ETS function in R applies automatic parameter selection to tune the model and select the best parameters such as the alpha smoothing parameter.

The simple ETS (SES) model was avoided for this study because that best fits data with no trend or seasonality. For this data, the following parameters were used with an ETS model:

- ETS_AAN or additive error and additive trend due to the pattern from the visualizations above (linear growth over time) and no seasonality (Holt Linear Trend model)
- ETS_AAM or multiplicative seasonality (the Holt-Winters Model, a version of ETS), due to the seasonality observed in the STL decomposition

4.2.2 Create Models

```
# create model fit objects for SNaïve, Naive, and ETS models
price_fit <- train_data |>
  model(
    Seasonal_naive = SNAIVE(Price),
    Naive = NAIVE(Price),
    Drift = RW(Price ~ drift()),
    ETS_AAN = ETS(Price ~ error("A") + trend("A") + season("N")),
    ETS_AAM = ETS(Price ~ error("A") + trend("A") + season("M"))
  )
```

```
# create an ensemble model
price_fit <- price_fit |>
  mutate(ensemble = (Seasonal_naive + Naive + Drift + ETS_AAN + ETS_AAM) / 5)
```

Then, a fable, or a forecast table, with point forecasts and distributions was created. This created forecasts for the 5 models using the fitted objects on training data and then created forecasts for the length of the test dataset. These forecasts were then compared with actual results from the test data.

4.2.3 Create Forecasts

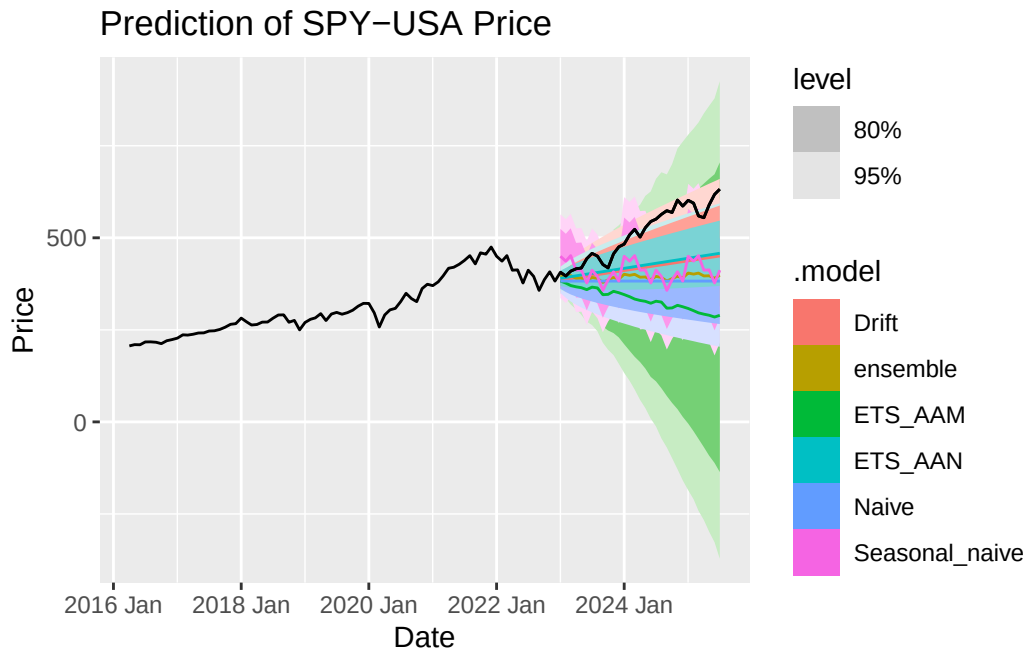
```
# create forecasts
forecasts <- price_fit |>
  forecast(h = nrow(test_data))
# forecast next 12 observations ie months
```

4.3 Model Evaluation

The model forecasts were then visualized, and key validation metrics such as RMSE, MAPE, and MAE were assessed. Additionally, residual diagnostic checks were conducted. This approach is effective for judging the efficacy of the models' fit to the data.

4.3.1 Forecast Plot

```
# create forecast plot
suppressMessages({forecasts |>
  autoplot(filter_index(data)) +
  autolayer(test_data, color = "black") +
  labs(y = "Price", title = "Prediction of SPY-USA Price")
})
```



4.3.2 Performance Metrics

To evaluate the models, RMSE, MAE, and MAPE, which are standard benchmarks in forecasting literature, were utilized. RMSE is particularly valuable here because it disproportionately penalizes large forecast misses, helping identify which models are most robust against the sudden “shocks” typical of the S&P 500. While MAE provides a measure of the typical daily dollar miss, MAPE offers a scale-independent percentage that is easier to interpret across different price levels.

```
# pull key performance metrics
accuracy(price_fit) |>
  arrange(.model) |>
  select(.model, .type, RMSE, MAE, MAPE) |>
  knitr::kable(digits = 2)
```

.model	.type	RMSE	MAE	MAPE
Drift	Training	16.19	11.78	3.55
ETS_AAM	Training	15.11	11.54	3.62
ETS_AAN	Training	15.70	11.45	3.46
Naive	Training	16.34	12.32	3.72
Seasonal_naive	Training	57.93	46.75	13.27
ensemble	Training	19.60	15.84	4.67

.model	.type	RMSE	MAE	MAPE
--------	-------	------	-----	------

The ETS_AAN (Holt’s Linear Trend) model emerged as the superior choice, achieving the lowest typical error with a MAPE of 3.46%. While the Drift and ETS_AAM models followed closely (3.55% and 3.62% MAPE, respectively), the ETS_AAN model was most effective at balancing the long-term upward trajectory with recent price action. As expected, the Seasonal Naïve model was the worst performer with a 13.27% MAPE, confirming that year-over-year cycles are not a reliable predictor for this data.

The visualization confirms the ETS_AAN and Drift forecasts both captured the upward momentum of the test data, while the Naïve model failed by assuming flat future prices. Though the ETS_AAM model attempted to capture a seasonal pattern, its higher error rates suggest it was overfitting to noise. By including high-performing trend models, the Ensemble model achieved a respectable 4.67% MAPE, significantly mitigating the negative influence of the Seasonal Naïve baseline.

4.3.3 Residual Diagnostics

To conduct residual diagnostic checks and further validate model performance, a Ljung-Box test was run to determine p-values for the different models and detect remaining autocorrelations in the residuals, whose presence would indicate a leftover pattern not captured by the model. For a model to be considered a strong fit, its residuals should behave as white noise, meaning all systematic patterns have been successfully captured and only random error remains.

```
# save augmented data
diag_info <- augment(price_fit)

# run ljung-box test for residual autocorrelation one
price_fit |>
  augment() |>
  features(.innov, ljung_box, lag = 10) |>
  knitr::kable(digits = 3,
               caption = "Ljung-Box Test for Residual Autocorrelation")
```

Table 3: Ljung-Box Test for Residual Autocorrelation

.model	lb_stat	lb_pvalue
Drift	15.501	0.115
ETS_AAM	14.313	0.159

.model	lb_stat	lb_pvalue
ETS_AAN	10.810	0.373
Naive	15.501	0.115
Seasonal_naive	160.822	0.000
ensemble	24.863	0.006

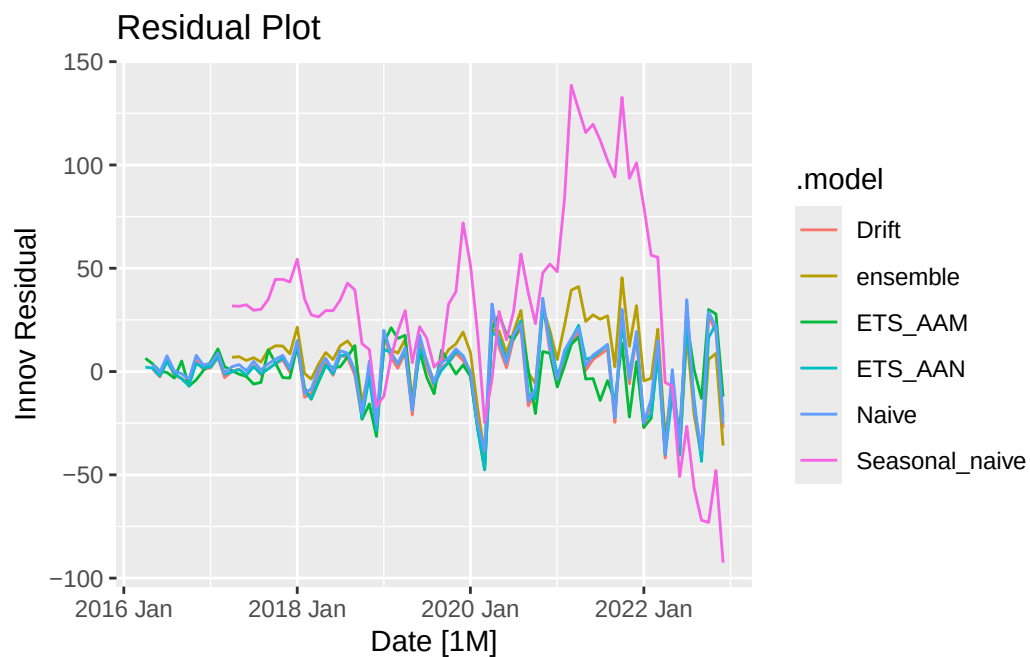
The Ljung-Box test results confirm that the ETS and naïve models successfully capture the underlying dynamics of the SPY-USA price. With p-values of 0.37 and 0.12, we can conclude that these models leave only white noise residuals. This confirms that the residuals are uncorrelated, and there is no information left in the residuals that should be used in computing forecasts. In contrast, the seasonal naïve model ($p < 0.01$) fails to account for the trend, resulting in significant residual autocorrelation.

4.3.4 Innovation Residuals

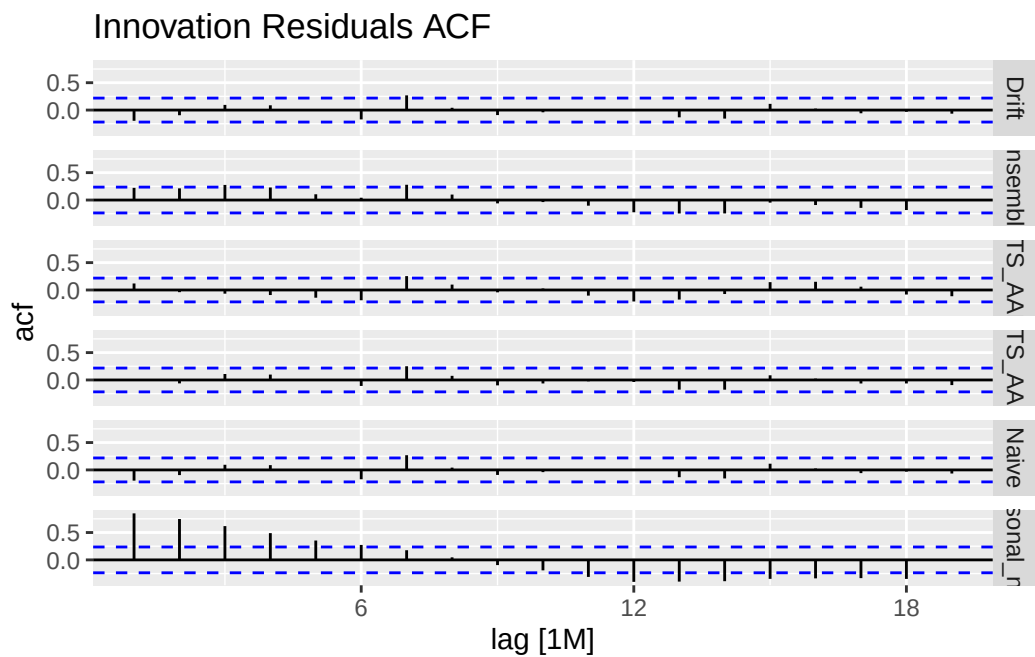
Innovation residuals are the error term, often analyzed to see if the models meet critical assumptions, like heteroscedasticity. At a minimum, the innovation residuals should have a constant variance and be normally distributed.

```
# check innovation residuals
diag_info <- diag_info |>
  filter(!is.na(.innov))

# create residual plot
autoplot(diag_info, .innov) +
  labs(title = "Residual Plot", y = "Innov Residual")
```



```
# run ACF on residuals
diag_info |>
  ACF(.innov) |>
  autoplot() +
  labs(title = "Innovation Residuals ACF")
```



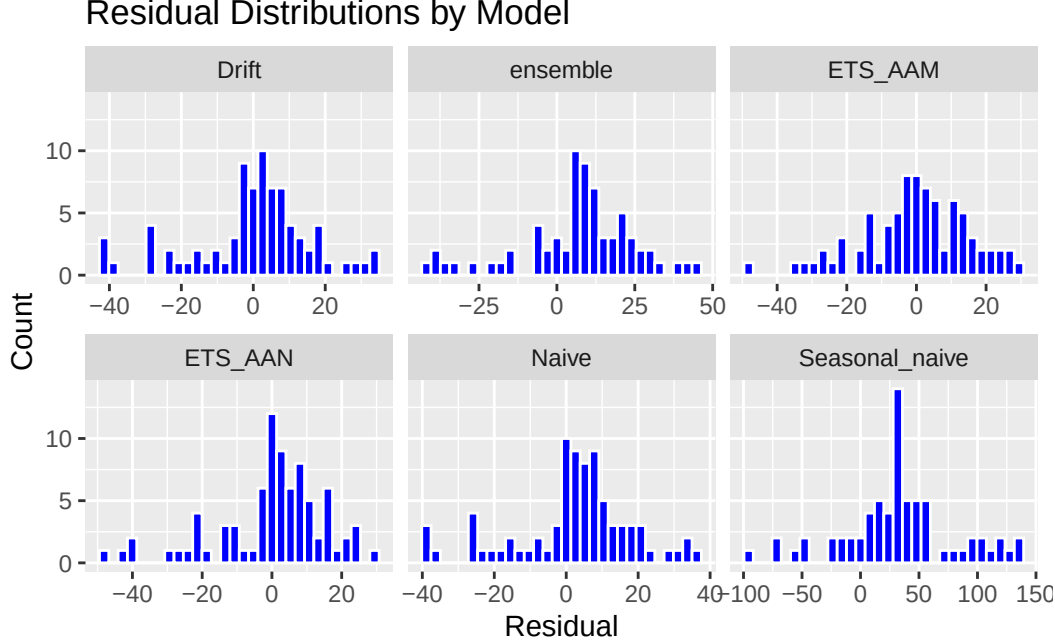
The innovative residual plot showed that seasonal naïve deviated from zero the most, indicating it has the greatest error of the models. Residuals should hover around zero and have a roughly constant spread. Most model innovation residuals hover around zero for the forecast period, except for seasonal naïve which exemplifies this strongest with its downward trend. This indicates bias in this model where there's possibly of overfitting a pattern on noise rather than capturing a true, predictable pattern.

The results of the autocorrelation function plot shows that most residuals stayed within acceptable bounds of autocorrelation, except for seasonal naïve. All other innovation residuals are generally within the accepted ranges, indicating the residuals behave like white noise and there is not information missing from the model causing systematic errors. The ensemble model stays within the bounds but shows similar behavior from the seasonal naïve data, likely being influenced by this model in its averaging calculations.

More holistic residual checks can be performed on each model to see the distribution of the residuals. This shows whether the mean of the residuals is zero.

4.3.5 Residual Histogram

```
# create residual histogram plot
diag_info |>
  filter(!is.na(.resid)) |>
  ggplot(aes(x = .resid)) +
  geom_histogram(bins = 30, fill = "blue", color = "white") +
  facet_wrap(~.model, scales = "free_x") +
  labs(
    x = "Residual",
    y = "Count",
    title = "Residual Distributions by Model"
  )
```



This visualization generally indicates that the assumption that the residuals are normally distributed is met.

Most of the innovation residuals are normally distributed, except for the ETS models. The seasonal naïve model appears to be normal but with a significant peak compared to the rest of the data.

5 Results and Analysis

From the forecast visualizations, accuracy metrics, and residual diagnostic checks, it's clear which models performed well and best fit the SPY-USA closing price pattern.

It can be argued that the ETS_AAN model, is the best performing model. The ETS model framework is the best fit visually, while also having the lowest typical error (MAE/MAPE). This model handles the upward trend gracefully, and avoids the unnecessary complexity of the seasonal model.

ETS models are effective because they can capture the baseline level pattern, but also incorporate trend or seasonality (or both) witnessed in the data. Exponential smoothing involves the weighted averaging of past observations where the weights decay exponentially with time. ETS models are flexible to include trend and seasonality and allows for explicit modeling of errors and uncertainty (Lee, 2025). The ETS model, incorporating various smoothing techniques into the state space framework, allows the model to adapt to data variability. The core idea with these models is to balance the trade-off between model flexibility and overfitting.

For the SPY-USA, it's clear an ETS model focusing on Trend + Noise (the remainder) beats a model looking for Seasonality. The seasonal naïve model tried to capture effects that weren't present in the raw data (though they appeared present in the STL decomposed data). This model was an inappropriate method to forecast stock price data because, while it captured some historical variation, it returned higher errors compared to other models. This is expected with stock price data that does not exhibit much seasonality.

The drift and naïve models are all fairly suited for financial data following a drift or random walk pattern, however the lower MAPE error and better visual fit from the ETS_AAN model helps prove that simply "guessing yesterday's price" is slightly less accurate than actually modeling the upward trend. The naïve model does not capture any sort of trend and forecasts the previous observation's value, while the drift incorporates this upward trend and is slightly more accurate. While the naïve model didn't capture the patterns found in the data as well as the ETS and drift models, it did serve as an effective comparative baseline. The ensemble model performance was in alignment with the performance of the models it encompasses and shows that the models tested for this forecasting exercise were generally well-suited to the data.

6 Discussion

The forecasting process has varied approaches, with different parameters that can be applied to fit the data and capture its events and patterns with accuracy.

With all these options, it's important to consider the data structure and behavior to pick the best model, and its parameters to capture data patterns like trends and seasonality. In comparing different models, it's also helpful to use simple baseline models, such as naïve models or seasonal naïve models, when appropriate.

What this study does is consider a popular financial investment, such as the SPY-USA ETF that tracks the S&P 500 and the different models that attempt to effectively forecast performance. This study focused on more obtainable, low computational forecasting methods for tech savvy and statistically minded investors. This study can be generalized to many American and global investors who want to benefit from investing in the S&P 500. It also explores an ensemble model, a model mentioned as an area of further exploration in related literature.

There are some limitations and challenges in this study, however. It doesn't consider more advanced machine learning or deep learning models that other studies consider. While it does explore ensemble methods that average results from many models, it doesn't include hybrid models that might have produced better results, like ETS_AAN and LSTM hybrid. This study is also limited to a single temporal variable, closing price, and does not account for exogenous factors that influence markets and ETF performance that are challenging to model, such as geopolitical factors.

Investors should consider simple and moderately complex models like ETS and drift as information to incorporate into investment decisions. Investors should not rely solely on these forecasts as they don't model stock market patterns with complete certainty, but they should pair these techniques with judgmental forecasting to make holistic investment decisions.

7 Conclusion

This study compares forecasting methods (naïve, seasonal naïve, ETS, drift and an ensemble), on 10 years of monthly SPY-USA data. Overall, the additive ETS model (Holt Linear) and the drift model delivered the smallest forecast errors (MAPE) and residuals that met key statistical assumptions. The seasonal naïve approach under-predicted the strong upward trend and exhibited autocorrelated residuals, making it the least reliable model. The ensemble model offered modest gains over its stronger competitors but was still negatively influenced by the seasonal naïve model bias.

Often, simple models suffice when it comes to financial instrument forecasting. Naïve forecasts provide a reasonable baseline while ETS models add flexibility by capturing trends. Also, it's import to match a model's form to its data structure so early exploratory analysis and decomposition efforts are crucial.

Limitations of this work include its focus on univariate historical prices and omission of exogenous factors (e.g. macroeconomic indicators or sentiment data), as well as more advanced machine- and deep-learning approaches. Future extensions might explore hybrid ETS-LSTM models, and incorporate technical or fundamental variables.

In practice, these forecasting tools can be “good-enough” to guide timing decisions while acknowledging that no model perfectly predicts market movements. By combining straightforward statistical methods with thoughtful domain judgment, investors can gain actionable insights without the cost or complexity of advanced methods.

8 References

- Ngwaba, C. A. (2025). Forecasting covered call exchange-traded funds (ETFs) using time series, machine learning, and deep learning models. *Journal of Risk and Financial Management*, 18(3), 120. <https://doi.org/10.3390/jrfm18030120>
- Pande, N. K., Kumar, A., & Gupta, A. K. (2025). Forecasting stock indices: Stochastic and artificial neural network models. *Computational Economics*, 65(4), 1937–1969. <https://doi.org/10.1007/s10614-024-10615-3>

- Shih, K.-H., Wang, Y.-H., Kao, I.-C., & Lai, F.-M. (2024). Forecasting ETF performance: A comparative study of deep learning models and the Fama-French three-factor model. *Mathematics*, 12(19), 3158. <https://doi.org/10.3390/math12193158>
- Yates, T. (2024, October 3). 4 ways to predict market performance. *Investopedia*. Retrieved July 29, 2025, from https://www.investopedia.com/articles/07/mean_reversion_martingale.asp
- FactSet. (n.d.). *SPY-US price history*. Retrieved July 29, 2025, from <https://my.apps.factset.com/workstation/navigator/company-security/price-history/SPY-US>
- Federal Reserve Bank of St. Louis. (n.d.). *Consumer Price Index for All Urban Consumers: All items (CPIAUCSL) and Core CPI (CPILFESL) [Data set]*. FRED. Retrieved July 29, 2025, from <https://fred.stlouisfed.org/graph/?id=CPIAUCSL,CPILFESL>
- Lee, S. (2025, March 18). 8 key steps to master exponential smoothing state space modeling. *Number Analytics*. Retrieved July 29, 2025, from <https://www.numberanalytics.com/blog/8-key-steps-to-master-exponential-smoothing-state-space-modeling>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/holt.html>